

Rank It, Then Ask It: Input Reranking for Maximizing the Performance of LLMs on Symmetric Tasks

Mohsen Dehghankar
University of Illinois Chicago
Chicago, IL, USA
mdehgh2@uic.edu

Abolfazl Asudeh
University of Illinois Chicago
Chicago, IL, USA
asudeh@uic.edu

Abstract

Large language models (LLMs) have quickly emerged as practical and versatile tools that provide new solutions for a wide range of domains. In this paper, we consider the application of LLMs on symmetric tasks where a query is asked on an (unordered) bag of elements. Examples of such tasks include answering aggregate queries on a dataset. In general, when the bag contains a large number of elements, LLMs tend to overlook some elements, leading to challenges in generating accurate responses to the query. LLMs receive their inputs as ordered sequences. On the other hand, the input of a symmetric task is not ordered; hence reordering should not affect the LLM's response.

Observing that LLMs are less likely to miss elements at certain positions of the input, we introduce the problem of LLM input reranking: to find a ranking of the input that maximizes the LLM's accuracy for the given query without making explicit assumptions about the query. Finding the optimal ranking requires identifying (i) the relevance of each input element for answering the query and (ii) the importance of each rank position for the LLM's attention. We develop algorithms for estimating these values efficiently utilizing a helper LLM. We conduct comprehensive experiments on different synthetic and real datasets to validate our proposal and to evaluate the effectiveness of our proposed algorithms. Our experiments confirm that our reranking approach improves the accuracy of the LLMs on symmetric tasks by up to 99% proximity to the optimum upper bound.

CCS Concepts

• **Computing methodologies** → *Natural language generation.*

Keywords

Large Language Models, Ranking, Input Reordering, Long-context

ACM Reference Format:

Mohsen Dehghankar and Abolfazl Asudeh. 2025. Rank It, Then Ask It: Input Reranking for Maximizing the Performance of LLMs on Symmetric Tasks. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3711896.3737099>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '25, Toronto, ON, Canada

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1454-2/2025/08
<https://doi.org/10.1145/3711896.3737099>

1 Introduction

Large Language Models (LLMs) have rapidly become invaluable tools, expanding their impact far beyond the realm of natural language processing. Tasks that have long been studied across various areas of computer science are now finding alternative solutions with LLMs.

In particular, LLMs can be used for symmetric [15] tasks where a query is issued on an input that takes the form of a bag (or multi-set) of elements. Dataset queries is an example of this type of task. To further clarify this, let us consider the following toy example.

Example 1: Consider a publications dataset in form of a CSV file, containing the information of papers published across various domains:

Authors	Title	Venue	Year
Alan Turing	Computing Machinery and Intelligence	Mind	1950
...	...	...	...

Suppose one is interested in finding out the number of publications in an "Operations Research" (OR) venue since 2010. They specify their query in the form of a prompt¹[how many papers were published in an operations research venue since 2010], and pass it alongside the CSV file to an LLM to find the answer.

Indeed, one can use the traditional approaches, such as implementing/generating an efficient code and executing it, or utilizing a text-to-SQL approach with a database (or data science packages), to answer the above query. However, many *ordinary users who view LLMs as omniscient* and may lack the required background knowledge, infrastructure, or interest may choose to directly pass their inquiries to an LLM, expecting it to know the solution. This is evident by the overwhelming popularity of LLMs as a revolutionary and versatile technology [39], motivating prior work to study LLMs for these settings [21, 22, 24, 28].²

Besides, LLMs may have learned the knowledge that helps for answering queries that otherwise are not "easy" to answer with the traditional approaches. For instance, answering Example 1 requires knowing which venues are in the OR category. As a result, a traditional approach for answering this query requires first acquiring a comprehensive list of all OR venues before being able to process it. On the other hand, using the semantic similarities, an LLM can directly process the query.

Examples of symmetric tasks are not limited to dataset queries. Particularly, Zhou et al. [15] study symmetric tasks on graph applications such as knowledge graphs, where, for example, passing the edges of a graph as the input elements, one can ask the LLM

¹A prompt is a textual instruction for the LLM.

²Please refer to Appendix A for an extended discussion of the related work.

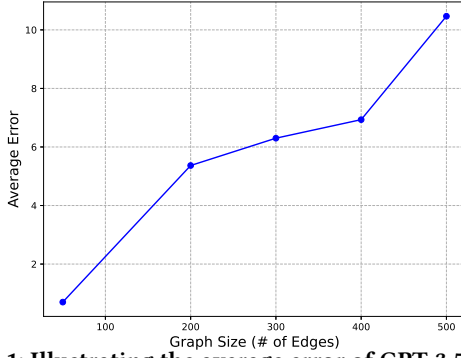


Figure 1: Illustrating the average error of GPT-3.5 Turbo on Graph Degree Task based on different input sizes. The error is the absolute difference between the real degree of a node (less than 20 in this case) and the reported degree by the LLM.

questions about node degrees (see Example 2). We refer to this as the Graph Degree Task.

LLMs follow a sequential randomized process to generate their outputs. As a result, their inputs are received and processed as ordered sequences. In particular, when the input is lengthy, LLMs are known to pay “less attention” to certain regions of the input, hence, struggling with retaining information from those regions. This leads to a performance degradation over extended inputs [23]. For example, Fig 1 shows the significant error of GPT-3.5 Turbo for Graph Degree Task when the graph size is more than 200.

On the other hand, the input of a symmetric task is an unordered bag of elements. As a result, the input elements can be freely reordered. This gives us the opportunity to *rerank the input elements* before passing it to the LLM for maximizing its accuracy – which is the research focus of this paper. Specifically, we introduce a reranking mechanism that (a) learns the so-called “exposure” of each rank position for an LLM during the preprocessing time. Then, at the query time, it (b) assesses the relevance of each input element to the given natural language query and (c) generates a reranking of the input that puts the relevant elements in high-exposure positions.

1.1 Summary of Contributions

In this work, we introduce the problem of LLM input reranking to find an ordering of the input that maximizes the accuracy of an LLM for symmetric tasks. To the best of our knowledge, we are the first to introduce and study this problem.

We propose a two-stage solution for the problem. First, we identify the exposures of the LLM to detect patterns of “forgetting” when processing large inputs. Second, we rank the input elements by estimating their relevance scores in relation to a query q , without requiring explicit knowledge of the task or the query itself. Our method enhances the effectiveness of LLMs in handling large and complex inputs in symmetric problem scenarios.

Our approach is *model-agnostic*, treating LLMs as black-box without requiring knowledge of their internal structures. It functions as a *ready-to-deploy wrapper* for LLM input reranking for accuracy maximization. Additionally, our method is *problem-agnostic*, optimizing a re-ranking function without explicit knowledge of the query or dataset, making it widely applicable across different domains.

We present extensive experimental evaluations across two distinct categories of tasks. First, in a Graph Degree Task, we demonstrate that by utilizing lightweight versions of open-source large language models, such as Llama 3.1, Qwen 2, and DeepSeek-Coder, we are able to estimate relevance scores and rank input elements with rank utility values approaching those of optimal solutions (see Table 1). Second, leveraging the obtained ranking information, we introduce a reranking approach that significantly enhances the accuracy of LLM outputs. *Our method improves performance by up to 99% proximity to the optimum solution*, as demonstrated in Table 3.

We also identify a notable distinction in the token retention patterns of two widely used commercial LLMs. For instance, we observe that GPT-3.5 Turbo demonstrates a stronger tendency to retain tokens positioned at the beginning of the prompt, whereas in GPT-4o Mini, tokens located in the middle are more likely to be remembered by the model (Fig. 3).

Additionally, we conducted experiments on other tasks, such as database query answering with real-world datasets, and observed comparable improvements in the output of the LLMs after reranking the input. More details on the experiments and more empirical results are provided in the Technical Report [9].

1.2 Paper Organization

The rest of the paper is organized as follows: we begin our technical discussions in Section 2, where we introduce the necessary notations and formalize our input reranking problem. Next, we present our algorithm for estimating the relevance of the input elements to the given query, in Section 3. In Section 4, we discuss our approach for learning the exposure values for an LLM for each rank position. Our experimental evaluations are provided in Section 5, followed by the discussion and conclusion. Due to the space constraints, the extended related work is provided in the Appendix.

2 Preliminaries

In this section, we formally define the problem and the specific notations that we use in the following sections.

2.1 Query Model

We assume that each task T is a pair $(\mathcal{I}, q)$. Where $\mathcal{I}$ is a (large) bag of elements $\{e_1, e_2, \dots, e_n\}$ and q is a query in natural language about $\mathcal{I}$. For example, for the Graph Degree Task, $\mathcal{I}$ is the list of edges with any order and q is a natural language question about the graph, like “What is the degree of node 10?”. We assume that $\mathcal{I}$ and q are given to the LLM as two different prompts in the same context. That is, we first provide the list to the LLM, then we ask about the query q . More details on the implementation is discussed in Section 5.

2.2 LLM Model

We assume that we have API access to a black-box LLM $\mathcal{L}$. For a task $T = (\mathcal{I}, q)$ as defined in the previous section, $\mathcal{L}(\mathcal{I}, q)$ is the response of the LLM. The output error is defined as:

$$\varepsilon_{\mathcal{L}}(\mathcal{I}, q) = \Delta[\mathcal{L}(\mathcal{I}, q), \mathcal{O}(\mathcal{I}, q)]$$

Where $O(\mathcal{I}, q)$ is defined as the correct output of the task, and Δ is a function that measures the distance between the LLM results and the correct output. For example, for the Graph Degree Task, Δ is the absolute difference between the reported degree and the real degree of the node.

2.3 Problem Definition

Given a task $(\mathcal{I}, q)$ and a large language model $\mathcal{L}$, our objective is to rerank the elements in $\mathcal{I}$ with respect to the query q to minimize the expected error, denoted as $\mathbb{E}(\varepsilon_{\mathcal{L}}(\mathcal{I}, q))$. Specifically, from the set of all possible rankings Π , we seek to identify the reranking function $\pi^* : \{1, 2, \dots, n\} \rightarrow \{1, 2, \dots, n\}$ that rearranges each element $e_{\pi^*(i)}$ to a new position i in such a way that the response of the LLM exhibits an improvement in terms of reduced expected error.

$$\pi^* = \arg \min_{\pi \in \Pi} \mathbb{E}[\varepsilon_{\mathcal{L}}(\mathcal{I}_{\pi}, q)] \quad (1)$$

Our approach is task (and query) agnostic. In other words, we find the function π^* without using any **explicit knowledge** about the query or the task.

2.4 Solution Overview

We define a *utility* function on different orderings of $\mathcal{I}$. Based on the objective function in our Problem Definition (Section 2.3), the *utility* of a reranking function π should capture the expected error $\mathbb{E}[\varepsilon_{\mathcal{L}}(\mathcal{I}_{\pi}, q)]$.

Let us define a function $Rel_q : \mathcal{I} \rightarrow [0, 1]$ that captures the relevance of each element $e_i \in \mathcal{I}$ to the query q . That is, $Rel_q(e_i)$ is the relevance of e_i to the query q .

Also, let $\mathcal{X}_{\mathcal{L}}(i)$ denote the “exposure” of the position i in a ranked input to the LLM $\mathcal{L}$, i.e., the likelihood that the LLM will not miss an element in position i . Then, the expected *utility* of a ranking π of the input $\mathcal{I}$ is calculated as [31],

$$\mathbb{E}[\text{utility}(\pi|q)] = \sum_{i=1}^{|\mathcal{I}|} \mathbb{E}[\mathcal{X}_{\mathcal{L}}(i)] \cdot \mathbb{E}[Rel_q(e_{\pi(i)})] \quad (2)$$

Example 2: Consider the graph G , with 6 vertices and the following edges:

e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8	e_9	e_{10}
1	2	1	3	1	2	3	3	5	2
2	4	4	4	3	5	5	6	6	6

Also, let the exposure function be $\mathcal{X}_{\mathcal{L}}(i) = \frac{1}{i}$. Now let the query q be “compute the degree of v_1 ”. For edges incident to v_1 , $Rel_q(e_i) = 1$, while for the others the relevance is 0. As a result, the utility of the ranking $\pi = \{1, 2, \dots, 10\}$ is $\text{utility}(\pi|q) = 1 + \frac{1}{3} + \frac{1}{5} \approx 1.53$. Note that the ranking with maximum utility put e_1, e_3 , and e_5 at the beginning of the list, and has the utility of $1 + \frac{1}{2} + \frac{1}{3} \approx 1.83$.

The first step towards computing the utility of a ranking is to specify the exposure function $\mathcal{X}$. We note that the *exposure values* are LLM-specific, i.e., the exposure of a position in the ranking may vary based on the LLM at hand and the length of each element in the input $\mathcal{I}$. Therefore, during the preprocessing time, we need to estimate the exposure values for a given LLM. To do so, in Section 4,

Algorithm 1 Warm-up

Input: The list of elements $\mathcal{I}$, The query q , The helper LLM $\mathcal{H}$, Number of partitions m

Output: The list of relevance scores $\{Rel_q(e_i) \mid e_i \in \mathcal{I}\}$

```

1: function PSM( $\mathcal{I}, q, \mathcal{H}, m$ )
2:    $P \leftarrow \text{PARTITION}(\mathcal{I}, m)$   $\triangleright$  Partition  $\mathcal{I}$  into  $m$  chunks of equal size
3:   for  $P_j \in P$  do
4:      $R \leftarrow \mathcal{H}(P_j, q)$   $\triangleright$  Ask helper to give the relevant elements  $R$ 
5:     for  $e_i \in P_j$  do  $Rel_q(e_i) \leftarrow 1$  if  $e_i \in R$ , 0 otherwise
6:   Return  $\{Rel_q(e_i) \mid e_i \in \mathcal{I}\}$ 

```

we develop an approach based on sample tasks for which the query q , the input $\mathcal{I}$, the correct output O , and the $Rel_q(e_i), \forall e_i \in \mathcal{I}$ is known ahead of time.

After specifying the exposure values during the preprocessing time, we need to estimate the relevance values at the query time. Recall that q can be any natural language query on the input set $\mathcal{I}$. Given a task $T = (\mathcal{I}, q)$, in Section 3, we present our approach for estimating the relevance function Rel_q .

3 Estimating the Relevance

Each element $e_i \in \mathcal{I}$ is associated with an implicit relevance score that quantifies its degree of relevance to the query q . This relevance score is denoted as $Rel_q(e_i)$. For instance, in Example 2, the edges adjacent to the node v are more relevant to the query “compute the degree of node v ”.

However, computing the relevance values is challenging since the task T is not specified beforehand, and q can be any query on $\mathcal{I}$. To address this issue, in this section, we study the sub-problem of estimating the relevance score for each element $e_i \in \mathcal{I}$.

We utilize a helper LLM $\mathcal{H}$ for estimating the relevance values. $\mathcal{H}$ is a small open-source LLM that is relatively cheap to run and deploy. In our experiments, we compare different models as $\mathcal{H}$, like Gemma2 (9b) [36], Llama3.1 (8b) [11], Qwen2 (7b) [41], DeepSeek-Coder-v2 (16b) [43], and Mistral (7b) [18].

In the rest of this section, we explain two methods to estimate the relevance scores. First, we discuss a warm-up baseline that partitions the input elements $\mathcal{I}$ into smaller subsets and identifies the relevant and non-relevant elements within each partition. Next, we present our estimation approach by modeling the problem as a bipartite graph.

3.1 Warm-up: Partitioning the Input

Let us consider Example 2 once again. To find the relevant edges to the given query, one can partition the input elements $\mathcal{I}$ into smaller subsets and ask the helper LLM to find relevant edges in each subset.

Following this idea, the warm-up algorithm first partitions the input list $\mathcal{I}$ into m smaller subsets of size $l = \lceil \frac{n}{m} \rceil$, i.e., $\{P_1 = \{e_1, \dots, e_l\}, P_2 = \{e_{l+1}, \dots, e_{2l}\}, \dots, P_m = \{e_{(m-1)l+1}, \dots, e_{ml}\}\}$. Next, for each partition P_i , the algorithm asks the helper LLM $\mathcal{H}$ “what elements in $[P_i]$ are more relevant for answering the query q ?”. It then assigns the relevance score 1 to the returned elements and 0 to others. See Algorithm 1 for the more details.

3.2 Modeling of the Relevance Estimation as Bipartite Graph

While providing a baseline for estimating the relevance values, the warm-up algorithm suffers from multiple drawbacks. First, the scope of the relevance values generated by the warm-up algorithm is limited to binary. Second, given the randomized nature of the LLMs, the output generated by the warm-up algorithm may not be reliable, particularly given that we assume we have no prior knowledge about the task and the query at hand. Third, depending on the composition of a partition, the output may miss relevant elements or return less relevant ones. In other words, the helper LLM may over/under-estimate the relevance scores in each partition. As a result, changing an element's partition may impact its relevance-value estimation.

To address the first issue, one can ask the helper to directly estimate the relevance value of each element in each partition, but this would result in high variance and sometimes inconsistent scores. Alternatively, addressing the first and second issues is possible by collecting multiple responses for each partition – instead of relying on a single evaluation. The relevance value of each element is then the average of each individual estimation. This, however, does not resolve the third issue; hence the estimations may remain inaccurate. Addressing the third issue is possible by shuffling the input before each partitioning, but this may require a large number of evaluations to generate unbiased and accurate estimations for each element – making the relevance estimation process costly.

Instead, our goal is to obtain accurate estimations with a *small number of evaluations per element*.

Let us now make a connection to the peer-reviewing process [25], in which a small number of reviewers review each paper, while various reviewers may generally provide higher/lower scores for the papers they review. Similarly, we can view each element as a paper and each relevance-value estimation for the elements in a partition as the scores provided by a reviewer.

Inspired by this connection, we devise a similar process for relevance-value estimation. Specifically, we randomly shuffle the input list $\mathcal{I}$ a total of σ times (for a small value of σ) to get the shuffled lists $\{\mathcal{I}_1, \mathcal{I}_2, \dots, \mathcal{I}_\sigma\}$. Subsequently, we partition each shuffled list into m equal-size subsets, as in the warm-up algorithm. This ensures that each evaluation of an element will be with a different set of elements, minimizing the partition composition impact on the final evaluations. Let $P_{i,k}$ denote the k^{th} partition of $\mathcal{I}_i$ where $i \leq \sigma$ and $k \leq m$. We ask the helper LLM $\mathcal{H}$ to give us a categorized score (e.g., from one to five) for each element inside each of these $P_{i,k}$ partitions. We index the score-evaluations to all partitions as $\{\mathcal{E}_1, \mathcal{E}_2, \dots, \mathcal{E}_{\sigma m}\}$ (the evaluations obtained for $P_{i,k}$ is $\mathcal{E}_{(i-1)m+k}$). Each evaluation can be considered a potentially biased (i.e., over/under-estimated) set of scores assigned to a set of elements.

For an element e_i , let $S_i = \text{Rel}_Q(e_i)$. Assume we could obtain a collection of σ unbiased evaluations $\{w_{i,1}^o, w_{i,2}^o, \dots, w_{i,\sigma}^o\}$ for e_i . Then, each $w_{i,j}^o$ would be viewed as a random variable taken from a distribution with mean S_i , i.e., $\mathbb{E}[w_{i,j}^o] = S_i$. Then $\bar{S}_i = \frac{1}{m} \sum_{j=1}^{\sigma} w_{i,j}^o$ would be an unbiased estimation for S_i .

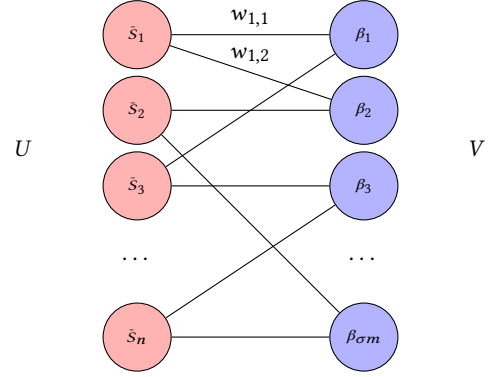


Figure 2: An example of the bipartite representation of evaluations. Each node in U represents an element in $\mathcal{I}$ and its final score. Each node in V represents one evaluation done on a partition $P_{i,k}$ from a shuffled list E_i and the associated bias with that. The weights on the edges are the scores assigned by the helper LLM $\mathcal{H}$ to elements.

Let $w_{i,j}$ be the evaluation score of e_i in $\mathcal{E}_j$. We assume that each evaluation $\mathcal{E}_j$ equally over/under estimates the evaluation scores for all elements with an unknown but constant coefficient β_j . That is, for all e_i evaluated in $\mathcal{E}_j$, $w_{i,j}^o = \frac{1}{\beta_j} w_{i,j}$. Had we known the bias values $\{\beta_1, \dots, \beta_j\}$, we could estimate the relevance score S_i of the element e_i as $\bar{S}_i = \frac{1}{\sigma} \sum_{e_i \in \mathcal{E}_j} \frac{1}{\beta_j} w_{i,j}$.

In order to find the bias coefficients for each of the evaluations, we build a bipartite graph $\mathcal{G}$, which we call the evaluation graph.

DEFINITION 1 (BIPARTITE EVALUATION GRAPH). Consider the weighted bipartite graph $\mathcal{G}(U, V, E)$. U contains n nodes, each representing an element $e_i \in \mathcal{I}$. V is a set of $\sigma \cdot m$ nodes, representing the evaluation outputs $\mathcal{E}_1, \dots, \mathcal{E}_{\sigma m}$. An edge (u_i, v_j) belongs to E iff $\mathcal{E}_j$ contains the element e_i , with its weight being equal to $w_{i,j}$ – the evaluation score $\mathcal{E}_j$ provides for e_i .

In the bipartite evaluation graph, the degree of each node $u_i \in U$ is $\deg(u_i) = \sigma$, while the degree of each node $v_j \in V$ is $\deg(v_j) = \lceil \frac{n}{m} \rceil$ (size of each partition). An illustration of the graph $\mathcal{G}$ is shown in Figure 2.

Let us associate each node $u_i \in U$ with the weight $\bar{S}_i$ and each node $v_j \in V$ with the weight β_j . Then, the following equations hold:

$$\begin{aligned} \bar{S}_i &= \frac{1}{\sigma} \sum_{(u_i, v_j) \in E} \frac{w_{i,j}}{\beta_j}, \quad \forall u_i \in U \\ \beta_j &= \frac{1}{\lceil \frac{n}{m} \rceil} \sum_{(u_i, v_j) \in E} \frac{w_{i,j}}{\bar{S}_i}, \quad \forall v_j \in V \end{aligned} \quad (3)$$

Intuitively, this equation gives an estimator of the bias values by averaging the samples taken from individual evaluations.

Therefore, to estimate the bias coefficients, we develop an iterative numeric process that at every iteration updates the node weights β_j (resp., $\bar{S}_i$) based on its current estimates of the weights $\bar{S}_i$ (resp., β_j) of the nodes connected to it.

The process starts by initializing all β_j values with 1 ($\beta_j^{(0)} = 1$). It then alternates between updating $\bar{S}_i$ values based on β_j values or we updating β_j using $\bar{S}_i$ values:

Algorithm 2 Bipartite

Input: The list of elements $\mathcal{I}$, The query q , The helper LLM $\mathcal{H}$, Number of partitions m , Number of shuffles σ

Output: The list of estimated relevance scores $\{Rel_q(e_i) \mid e_i \in \mathcal{I}\}$

```

1: function BEM( $\mathcal{I}, q, \mathcal{H}, m, \sigma$ )
2:    $G \leftarrow$  Initialize the bipartite graph
3:   for  $1 \leq i \leq \sigma$  do
4:      $\mathcal{I}_i \leftarrow$  Random shuffle  $\mathcal{I}$ 
5:      $P_i \leftarrow$  Partition  $\mathcal{I}_i$  into  $m$  chunks
6:     for  $P_{i,k} \in P_i$  do
7:        $\mathcal{E} \leftarrow \mathcal{H}(P_{i,k})$  ▷ Ask helper to score this chunk
8:       Update  $G$  ▷ Add edges  $w_{i,j}$  based on observed scores.
9:   Initialize  $\beta_j$  values inside  $G$ 
10:  while didn't converge do
11:    Update  $\tilde{S}_i$  and  $\beta_j$  values based on Equations 4
12:  Return  $\{\tilde{S}_1, \tilde{S}_2, \dots, \tilde{S}_n\}$ 

```

$$\begin{aligned}\tilde{S}_i^{(T)} &= \frac{1}{\sigma} \sum_{(u_i, v_j) \in E} \frac{w_{i,j}}{\beta_j^{(T-1)}}, \forall u_i \in U \\ \beta_j^{(T+1)} &= \frac{1}{\lceil \frac{n}{m} \rceil} \sum_{(u_i, v_j) \in E} \frac{w_{i,j}}{\tilde{S}_i^{(T)}}, \forall u_i \in U\end{aligned}\quad (4)$$

Where $\tilde{S}_i^{(T)}$ (resp., $\beta_j^{(T)}$) is the estimated relevance score (resp. bias coefficient) at time step T . A pseudo-code of this method is presented in Algorithm 2.

THEOREM 1. *The process described in Equation 4 would eventually converge.*

PROOF. (Proof sketch) This process resembles the process applied to a positive semi-definite matrix, where, in each step, we alternately normalize all rows and then all columns. This procedure preserves the eigenvalues of the matrix. According to Sinkhorn's theorem, such a process converges. A formal proof is provided in Appendix B. $\square$

The above algorithm is run as the preprocessing phase of our method. As part of this step (Algorithm 2), we issue API calls to the helper LLMs $\mathcal{H}$, lightweight models designed for efficient execution during preprocessing. The total number of such calls is σm , corresponding to the total number of evaluations, as shown in Figure 2, where σ denotes the number of shuffles and m represents the number of items in each partition.

4 Pre-processing: Exposure Discovery

So far, we studied the relevance estimation of the input elements to a given query. The remaining information for the LLM-input reranking based on Equation 2 is to compute the exposure values for different rank positions. The exposure values are LLM-dependent, and hence, we compute those during the preprocessing phase for each large language model $\mathcal{L}$.

The exposure value of the position i in the ranking, i.e., $\mathcal{X}_{\mathcal{L}}(i)$, represents the likelihood that the model misses each token in the input prompt as a function of its position i . Throughout this section, i will be used to indicate the position of a **token** within the input prompt to $\mathcal{L}$. In the end, we shall explore how this information can

be utilized for the general list $\mathcal{I}$, where each element $e_i \in \mathcal{I}$ is not necessarily a token.

In order to estimate the exposure values, we consider a sample set of predefined tasks, each consisting a query q and the input elements $\mathcal{I} = [t_1, t_2, \dots, t_n]$ (consisting of n tokens arranged in sequential order). The task samples can be viewed as the training data we use to learn the exposure values. Note that for each of the tasks, we already know the ground-truth relevance values $Rel_q(\cdot)$ and the output of the query, i.e., $O(\mathcal{I}, q)$. Hence, for an output generated by the LLM, we can calculate the output error $\epsilon_{\mathcal{L}}(\mathcal{I}, q)$. Let $Rel_q(t_i)$ be either 0 or 1 for each token in $\mathcal{I}^3$. Our goal is to estimate a set of unknown values $\{\mathcal{X}_{\mathcal{L}}(i) \mid i \leq n\}$.

After passing the task $(\mathcal{I}, q)$ to $\mathcal{L}$ with a specific ordering of $\mathcal{I}$, the error of the output provides aggregate information about the values $\mathcal{X}_{\mathcal{L}}(i)$ based on the relevance scores. We model the relation between the error and the exposures as,

$$\frac{1}{\mathbb{E}[\epsilon_{\mathcal{L}}(\mathcal{I}, q)]} \propto \frac{1}{n} \sum_{i=1}^n (\mathcal{X}_{\mathcal{L}}(i) \cdot Rel_q(t_i)) \quad (5)$$

In other words, the inverse of the error is directly proportional to the rank utilization of the items within $\mathcal{I}$. The higher utilization implies that items of greater relevance are positioned in more exposed locations ($\mathcal{X}_{\mathcal{L}}(i)$). Consequently, this results in a relatively smaller output error of the LLM.

In the Appendix C, we provide the details of estimating the exposures $\mathcal{X}$ and a confidence bound on the estimation. Finally, a discussion of the practical details for the implementation is provided there.

5 Experiments

In this section, we assess the practical applicability of our results through experiments conducted on both real and synthetic datasets. Specifically, we evaluate three categories of tasks. The first category is similar to Example 1, where we have a dataset of published papers and ask for a query about the papers. The second task focuses on the Graph Degree Task (Example 2), where a graph is constructed, and its edges are provided as input to the LLM, denoted as $\mathcal{I}$. A query q is then issued, requesting the degree of a specific node within the graph. The final category involves answering queries about structured datasets. In these tasks, we supply a table to the LLM ($\mathcal{I}$) and ask an aggregation query (q), expressed in natural language, about the table. We try different real-world datasets for this category. All experiments were conducted on a local server equipped with 128 GB of memory, 32 CPU cores, and two NVIDIA GeForce RTX 2080 Ti GPUs. The code is also accessible through this repository.

In the subsequent subsections, we begin by detailing the process of exposure discovery. Following this, we provide an in-depth discussion of the tasks, including the methods employed to evaluate the LLMs. Next, we analyze the ranking performance of various open-source models (referred to as helpers) on these tasks. Finally, we compare the effectiveness of the different reranking techniques presented throughout this study. Due to the space limitations, the detail of the experiments, as well as the examination of the effectiveness of the exposure discovery, is provided in the Technical

³Check the experiments section (Section 5) for practical examples.

Report [9]. Finally, a discussion on the advantages and limitations of our approaches is provided in Section 6.

5.1 Exposure Discovery

In the pre-processing phase, we determine the exposure of each input token position to the LLM denoted as $X_{\mathcal{L}}(i)$. For this experiment, we utilize two widely adopted closed-source LLMs: **GPT-3.5 Turbo** and **GPT-4o Mini**, and two open-source LLMs: **Gemma-2:27B** [36] and **Phi-4:14B** [1]. To measure token exposure values, as outlined in Section 4, we construct a synthetic task denoted as $(\mathcal{I}, q)$, where each element within $\mathcal{I}$ corresponds to an individual token. The query q prompts the LLM to report the frequency of occurrence for each token in the input set $\mathcal{I}$. The LLM's output is represented as a key-value list, where each key corresponds to a token, and the associated value indicates the number of times that token was identified within the input.

We systematically reposition various tokens, including their repeated occurrences across consecutive positions (aka windows) within the input $\mathcal{I}$, to assess the extent to which each position is exposed to the LLM. We estimate the exposure levels by conducting multiple sampling iterations, as illustrated in Figure 3. The errors are computed as the absolute difference between the actual repeats of each token and the corresponding LLM prediction, averaged across all unique tokens. The exposure is modeled as the inverse of error at each position.

Based on Figure 3, GPT-3.5 Turbo tends to focus more on the beginning of a long prompt, while it is more likely to forget the later tokens. In contrast, GPT-4o Mini shows a different pattern, where it is more likely to forget tokens at the start and the end while keeping those in the middle. As shown in the Technical Report [9], Phi-4 behaves similarly to GPT-4o Mini, mainly retaining the middle part of the prompt. On the other hand, Gemma-2 remembers tokens at the end of the prompt but tends to forget the beginning.

We leverage the exposures identified in this subsection as a preprocessing step to address the subsequent tasks discussed in the following subsection.

5.2 Tasks Setup

In this section, we present a detailed overview of the task setups and the datasets utilized.

5.2.1 Publication Venue Task. Following Example 1, we utilized the DBLP ACM dataset [29], with approximately 132,000 entries. This dataset provides a comprehensive list of research papers, including attributes such as titles, authors, and the journals in which the papers were published. It is widely used for entity resolution tasks. For our study, we extracted a smaller sample of rows from this dataset. The task involved passing the sampled DataFrame to the LLM as input and querying it with questions such as, "How many papers were published in physics-related journals?" to determine the number of papers relevant to a given research topic.

5.2.2 Graph Degree Task. In this task, we generate a random graph using the Erdős-Rényi model [14], where the edges represent the set of elements, denoted as $\mathcal{I}$. The primary query q for this graph is: "What is the degree of a given node v ?" We systematically select various nodes v within the graph and evaluate the responses provided by an LLM to these queries. Since the generated graph is fully accessible, we can compute the ground-truth degree of each

queried node. To assess the accuracy of the LLM's responses, we calculate the error $\epsilon_{\mathcal{L}}$ as the absolute difference between the degree reported by the LLM and the ground-truth degree. We first pass the set of edges to the LLM and then ask the query in a different prompt message but in the same context. We selected graph sizes up to 500 edges to generate synthetic graphs to ensure that all edges could be represented within a single prompt when inputting them into LLMs, thereby adhering to the models' token limits.

5.2.3 Dataset Query Task. Similar to the publication venue task, in this task, we provide a dataset table as input to the LLM and ask *aggregation queries* in natural language. For instance, a query might ask, "How many rows contain the value in the 'col' column equal to 'value'?"

We utilize *three real-world datasets* for this study. The IMDB Movies dataset [26] provides information on the top 1,000 movies listed on IMDB. We extract a subset containing 60 rows from this dataset and formulate a query regarding the number of movies with a rating of $Rating \geq 8.2$. The Adult Income dataset [2], which comprises data on approximately 48,000 individuals for income prediction tasks, is also employed. From this dataset, we sample 60 rows and pose a query about the number of individuals associated with a specific "workclass" category. Finally, the Open University Learning Analytics (OULAD) dataset [19] includes data on student enrollments across various courses. It contains 32K rows. For our task, we sample a subset of 100 rows and ask about the number of students enrolled in a particular course. These sample sizes are carefully selected to align with the token limits of LLMs⁴. The results for the Adults and OULAD are presented in the Technical Report [9].

5.3 Analysis of the Ranking utility

In this subsection, we utilize different small-sizes open-source LLMs as helpers ($\mathcal{H}$) to estimate the relevance scores of elements to the query (Rel_q). We use five different models: **DeepSeek-Coder-v2:16B** [43], **Gemma2:9B** [36], **Llama3.1:8B** [11], **Qwen2:7B** [41], and **Mistral:7B** [18]. This selection, while random within the range of small models (approximately 7B–8B parameters), ensures coverage of diverse and widely used open-source models.

We compare four ranking methods in this experiment. The first method, **Warm-up**, as described in Section 3.1, involves querying each partition of the input and subsequently splitting it. The second method, **Bipartite**, detailed in Section 3.2, utilizes Bipartite Graph Modeling to estimate relevance scores. To establish a lower bound for performance comparison, we use the **Random** method, which involves randomly shuffling all the input elements within the set $\mathcal{I}$ and passing them to the language model, $\mathcal{L}$. We expect this method to give us less accurate results. As an upper bound, we employ the **Optimum** method, where elements are completely sorted based on explicit knowledge of the task to maximize relevance to the query. This approach provides a theoretical upper limit for ranking utility, and we expect it to give us the best answer. For example, for the

⁴GPT-3.5-Turbo has a token limit of 4,096. For this model, we selected 60 rows from the ADULT dataset, which resulted in almost 3,244 tokens. For GPT-4o-mini, which supports a significantly larger context window of 128k tokens, we selected a correspondingly larger number of rows.

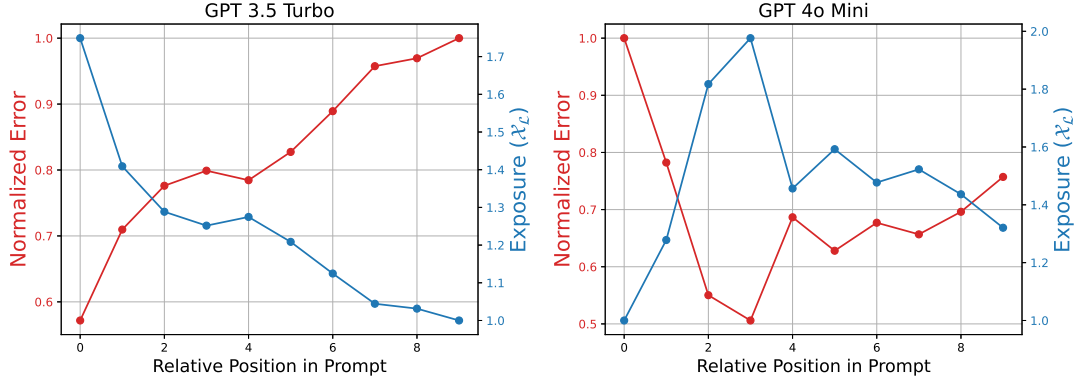


Figure 3: Relative exposure values for different LLMs. The red plot represents the normalized error observed when placing relevant data at specific positions within a long prompt, averaged over 100 runs. The prompt length is 2000 tokens for Gemma2:27B and 1000 for others. In our formulation, the inverse of the average error at each position is proportional to the exposure $\chi_{\mathcal{L}}(i)$. Higher error at a given location indicates lower exposure at that index.

Table 1: Ranking utility comparison across algorithms and helper models. Each point is an average of 10 runs. The percentage values are the proximity of the numbers compared to the upper and lower bounds. The green (red) arrow indicates the closeness to the upper (lower) bounds. The closer the values in the table are to the Optimum, the better the results.

Algorithm	Synthetic Graph Task				
	DeepSeek-Coder-V2 (16B)	Gemma2 (9B)	Llama3.1 (8B)	Mistral (7B)	Qwen2 (7B)
Optimum (UB)	3.02 (100%) ↑	2.98 (100%) ↑	3.01 (100%) ↑	3.00 (100%) ↑	2.98 (100%) ↑
Bipartite	2.95 (97%) ↑	1.87 (58%)	2.22 (70%) ↑	2.49 (81%) ↑	1.03 (26%) ↓
Warm-up	0.67 (13%) ↓	2.58 (85%) ↑	1.70 (51%)	2.03 (63%) ↑	0.72 (15%) ↓
Random (LB)	0.31 (0%) ↓	0.31 (0%) ↓	0.32 (0%) ↓	0.33 (0%) ↓	0.32 (0%) ↓
	IMDB Dataset				
	DeepSeek-Coder-V2 (16B)	Gemma2 (9B)	Llama3.1 (8B)	Mistral (7B)	Qwen2 (7B)
Optimum (UB)	2.76 (100%) ↑	2.60 (100%) ↑	2.69 (100%) ↑	2.52 (100%) ↑	2.67 (100%) ↑
Bipartite	2.63 (94%) ↑	2.50 (95%) ↑	2.48 (90%) ↑	2.22 (84%) ↑	1.60 (48%)
Warm-up	1.30 (33%)	2.58 (99%) ↑	1.68 (52%)	2.22 (84%) ↑	1.50 (44%)
Random (LB)	0.57 (0%) ↓	0.48 (0%) ↓	0.58 (0%) ↓	0.55 (0%) ↓	0.58 (0%) ↓
	DBLP ACM Dataset				
	DeepSeek-Coder-V2 (16B)	Gemma2 (9B)	Llama3.1 (8B)	Mistral (7B)	Qwen2 (7B)
Optimum (UB)	3.27 (100%) ↑	3.35 (100%) ↑	3.35 (100%) ↑	3.29 (100%) ↑	3.36 (100%) ↑
Bipartite	2.16 (57%) ↑	2.76 (77%) ↑	1.68 (56%)	1.65 (26%) ↓	1.65 (32%)
Warm-Up	0.89 (8%) ↓	2.25 (58%)	1.45 (41%)	1.86 (35%)	1.36 (21%) ↓
Random (LB)	0.66 (0%) ↓	0.72 (0%) ↓	0.00 (0%) ↓	1.07 (0%) ↓	0.81 (0%) ↓

Graph Degree Task, we sort all the elements based on the fact that whether the edge contains the query node v or not.

Table 1 presents the comparison of ranking utilities across different models and methods (more results on additional datasets are illustrated in the Technical Report [9]). The Random and Optimum methods serve as lower and upper bounds, respectively, providing benchmarks for comparison independent of any specific helper model. Due to the variability introduced by sampling, slight differences in the ranking utilities across different models can occur

for these two methods, even though they do not use any helper models. Each value reported in the table represents the average of 10 independent runs.

The values represent the ranking utilities computed under the assumption that the exposure function is given by $\chi_{\mathcal{L}}(i) = \frac{1}{i}$. In other words, once each helper model assigns relevance scores, a corresponding reranking function is obtained based on the specific model and method applied. To evaluate and compare the performance of different models, we analyze their respective rerankings.

Specifically, we first sort the elements according to the reranking produced by each model. Within the sorted list, we then compute the utility associated with the truly relevant elements using exposure $\frac{1}{i}$. A higher utility value indicates that the relevant elements are positioned at higher ranks, reflecting that the reranking generated by the model assigns them higher estimated relevance scores.

The percentage values in the table indicate the **Proximity** of the results, which can be defined as below.

DEFINITION 2 (PROXIMITY). Let L be the lower bound and U be the upper bound for a given observed error x . The proximity $P(x)$ with respect to these bounds can be defined as:

$$P(x) = \frac{x - L}{U - L}, \quad \text{where } L \leq x \leq U.$$

Based on this observation, in most cases, the **Bipartite** approach estimates relevance scores nearly equivalent to those of the optimal solution. However, for certain models, such as Gemma2, the **Warm-up** algorithm yields better results. This is because the Bipartite method requires the model to generate a list of scores, and the resulting output from the model may not always align with expectations. In contrast, for models designed to perform well in coding tasks, such as DeepSeek-Coder-v2 and Llama3.1, the Bipartite graph approach effectively enhances the reranking of the prompt.

The impact of the iterative procedure described in Algorithm 2 is demonstrated in Table 2. This table compares the ranking utility obtained after running the iterative process to convergence with the utility achieved without applying the procedure, using the original (biased) model evaluations. As shown, the iterative refinement effectively reduces bias and leads to improved ranking utility.

Setting	DeepSeek-Coder-V2	Llama3.1
w/o convergence	0.84	1.96
after convergence	2.95	2.22

Table 2: The effect of iterative process (Algorithm 2) in the final ranking utility. ‘w/o convergence’ is the result of applying Equation 3 only once to achieve (biased) scores, while the second row shows the effect of using converged scores. The results are for the Synthetic Graph Task.

The number of API calls to the helper models is determined by the number of samples used in the experiment. In this experiment, we used 10 samples for each reported number. For the Bipartite method, we performed four shuffles, where each shuffle divided the input into chunks of size 20. Consequently, the total number of API calls is calculated as $4 \times (100/20)$, where 100 represents the length of the input prompt (i.e., the number of elements), resulting in 20 API calls per open-source model for each sample. Similarly, for the Warm-Up method, the total number of API calls remains 20, as we used a partition size of 5, leading to $100/5 = 20$ partitions.

5.4 Analysis of the Output Error

In this subsection, we compare the helper models and the methods in achieving the final goal which is enhancing the final output error from the LLM $\mathcal{L}$. The results for GPT-3.5 Turbo and GPT-4o Mini are presented in Tables 3b and 3a. For each helper model we

used shorter names: DC2 (DeepSeek-Coder-v2), G2 (Gemma2), L3.1 (Llama3.1), M (Mistral), and Q2 (Qwen2). In these tables, Optimum is a lower bound since it is the best result one can achieve.

Each value in these tables represents the average of 10 runs. The errors for each helper model on a given dataset are normalized to the range $[0, 1]$. In most cases, the final error for the model employing the Bipartite method is close to the optimal solution. However, certain helper models, such as Qwen2, perform poorly in reranking the prompt, resulting in errors comparable to those obtained from random shuffling of the input. We observe that the Adults dataset presents a relatively more challenging task in most cases. This increased difficulty arises from the larger number of columns in this dataset compared to others, with a significant portion of these columns consisting of string-type data. The prevalence of such features complicates the query-answering process for the LLM.

6 Discussions

In this section, we first examine the advantages of our method in practical applications, followed by a discussion of its limitations.

6.1 Advantages

In all the proposed models and methods, we have the assumption that the final large language model is treated as a black-box, meaning we have no explicit knowledge of its internal architecture and make no use of such details. Even the smaller helper models employed in our approach are more cost-effective language models, yet we similarly assume no access to or detailed understanding of their underlying structures. Consequently, our method can be regarded as a wrapper that complements any advancements in large language models and improvements in their accuracy. This approach is applicable to any given LLM to enhance its performance in addressing symmetric problems, functioning as an additional layer to increase overall accuracy. In general, without detailed knowledge of a model’s architecture, it is challenging to understand how it remembers different parts of the input. However, our approach aims to estimate the exposure of different input segments, allowing us to infer the recall patterns of a black-box LLM.

Additionally, we adopt an abstract approach from the problem perspective. Without requiring explicit knowledge of the specific problem or the query applied to the bag of elements, we aim to identify a re-ranking function that optimizes the final accuracy. This approach serves as a generalizable solution applicable to any symmetric problem (a query asked about a set or multi-set of elements).

The proposed model and algorithm for debiasing the LLM evaluations, known as bipartite evaluation, can be applied in any scenario where pointwise evaluations are performed on a set of objects and varying biases exist across different evaluations. More broadly, the approach is applicable to any problem that can be framed as a peer-review (peer-grading) process.

6.2 Limitations

As discussed in previous sections, our focus is on symmetric problems, where a set (or multi-set) of elements $\mathcal{I}$ is presented, and a query is asked about it. Our methods aim to re-rank $\mathcal{I}$ under this assumption and, therefore, are not applicable in cases where the

Table 3: Normalized Error ($\varepsilon_{\mathcal{L}}$) across algorithms and helper models. The errors are normalized for each helper model to align them in the interval $[0, 1]$. Each value is an average of 10 runs. The **green (**red**) arrow indicates the closeness to the lower (upper) bound. The closer the values in the table are to the Optimum, the better the results.**

(a) Output error results on GPT-4o Mini						(b) Output error results on GPT-3.5 Turbo					
Algorithm	Synthetic Graph Task					Algorithm	Synthetic Graph Task				
	DC2	G2	L3.1	M	Q2		DC2	G2	L3.1	M	Q2
Random (UB)	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑	Random (UB)	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑
Warm-up	0.98 ↑	0.12 ↓	0.99 ↑	0.5	0.72 ↑	Warm-up	0.72	0.35	0.63	0.90 ↑	0.58
Bipartite	0.12 ↓	0.63	0.68	0.12 ↓	0.28 ↓	Bipartite	0.09 ↓	0.59	0.37	0.14 ↓	0.30
Optimum (LB)	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓	Optimum (LB)	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓
	IMDB Dataset						IMDB Dataset				
	DC2	G2	L3.1	M	Q2		DC2	G2	L3.1	M	Q2
Random (UB)	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑	Random (UB)	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑
Warm-up	0.33	0.11 ↓	0.64	0.42	0.68	Warm-up	0.56	0.87 ↑	0.85 ↑	0.49	0.50
Bipartite	0.25 ↓	0.10 ↓	0.01 ↓	0.42	0.62	Bipartite	0.03 ↓	0.29 ↓	0.04 ↓	0.69	0.48
Optimum (LB)	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓	Optimum (LB)	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓
	DBLP ACM Dataset						DBLP ACM Dataset				
	DC2	G2	L3.1	M	Q2		DC2	G2	L3.1	M	Q2
Random (UB)	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑	Random (UB)	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑	1.00 ↑
Warm-up	0.46	0.77 ↑	1.00 ↑	0.25 ↓	0.89 ↑	Warm-up	0.45	0.57	0.71	0.47	0.84 ↑
Bipartite	0.37 ↓	0.24 ↓	0.71	0.27 ↓	0.58	Bipartite	0.23 ↓	0.45	0.31 ↓	0.58	0.77 ↑
Optimum (LB)	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓	Optimum (LB)	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓	0.00 ↓

input is an ordered list, and the ordering of elements plays a crucial role in the final response to the query.

Additionally, we assume that different elements have different relevance when answering the given query. Hence, placing the high-relevance elements in highly exposed positions within the input sequence leads to improved accuracy. Conversely, when all elements in the input set are nearly equally relevant to the query, re-ranking the input may result in only marginal accuracy improvements. Regardless of the sorting method, some highly relevant elements may still stay in less exposed locations, leading to potential degradation in the LLM’s ability to retain them effectively.

Finally, in a practical setting, the proposed approaches necessitate the use of a significantly smaller helper model in conjunction with the primary large language model. As a result, deploying these methods requires the additional deployment of a smaller model.

7 Conclusion

In this work, we addressed the challenge of handling lengthy input prompts for large language models within the context of *symmetric tasks*. We observe that while the performance of LLMs drops for large inputs, certain positions in the input are less likely to be missed by an LLM. Moreover, reordering the input elements of symmetric tasks does not logically affect the query outcome.

Following these observations, we introduced the LLM input reranking problem to reorder an input in a way that the accuracy of the LLM is maximized for the given query.

We proposed a two-step solution for this problem. First, during the preprocessing phase, we identify the exposure of ranking positions for a given LLM. Next, at query time, we estimate the relevance score of each element to the query. By combining these insights, we rerank the input and pass the reranked data to the LLM. For query relevance estimation, we introduced a method based on a bipartite graph modeling, with a performance to the optimal reranking in our experiments.

Our solutions treat both the tasks and the LLMs as black-box components, allowing for a high level of abstraction. As a result, those can serve as a wrapper layer on top of any of the existing or future LLMs for solving symmetric tasks.

Despite the contributions of this work, certain limitations remain. For instance, the deployment of smaller LLMs and their suboptimal performance in score estimation in certain cases present challenges. By addressing these limitations and exploring enhancements in the reranking approaches, future research has the potential to further advance the effectiveness and scalability of language models in real-world applications.

Acknowledgments

This work was supported in part by the National Science Foundation, Grant No. 2348919 and 2107290.

References

- [1] Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J. Hewett, Mojan Javaheripi, Piero Kauffmann, James R. Lee, Yin Tat Lee, Yuanzhi Li, Weishung Liu, Caio C. T. Mendes, Anh Nguyen, Eric Price, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Xin Wang, Rachel Ward, Yue Wu, Dingli Yu, Cyril Zhang, and Yi Zhang. 2024. Phi-4 Technical Report. *arXiv preprint arXiv:2412.08905* (2024). <https://arxiv.org/abs/2412.08905> Accessed: 2025-01-31.
- [2] Barry Becker and Ronny Kohavi. 1996. Adult. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5XW20>.
- [3] Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150* (2020).
- [4] Tom B Brown. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).
- [5] Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. 2023. Walking down the memory maze: Beyond context limit through interactive reading. *arXiv preprint arXiv:2310.05029* (2023).
- [6] Jiawei Chen, Hande Dong, Xiang Wang, Fuli Feng, Meng Wang, and Xiangnan He. 2023. Bias and debias in recommender system: A survey and future directions. *ACM Transactions on Information Systems* 41, 3 (2023), 1–39.
- [7] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. 2023. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595* (2023).
- [8] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509* (2019).
- [9] Mohsen Dehghankar and Abolfazl Asudeh. 2024. Rank It, Then Ask It: Input Reranking for Maximizing the Performance of LLMs on Symmetric Tasks. *arXiv:2412.00546* <https://arxiv.org/abs/2412.00546>
- [10] Daniel Deutsch, Rotem Dror, and Dan Roth. 2022. On the limitations of reference-free evaluations of generated text. *arXiv preprint arXiv:2210.12563* (2022).
- [11] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [12] Sana Ebrahimi, Kaiwen Chen, Abolfazl Asudeh, Gautam Das, and Nick Koudas. 2024. AXOLOTL: Fairness through Assisted Self-Debiasing of Large Language Model Outputs. *15th IEEE International Conference on Knowledge Graphs* (2024).
- [13] Sana Ebrahimi, Nima Shahbazi, and Abolfazl Asudeh. 2024. REQUAL-LM: Reliability and Equity through Aggregation in Large Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2024*. 549–560.
- [14] Paul Erdos, Alfréd Rényi, et al. 1960. On the evolution of random graphs. *Publ. math. inst. hung. acad. sci* 5, 1 (1960), 17–60.
- [15] Jianfei Gao, Yangze Zhou, Jincheng Zhou, and Bruno Ribeiro. 2023. Double equivariance for inductive link prediction for both new nodes and new relation types. *NeurIPS* (2023).
- [16] Junqing He, Kunhao Pan, Xiaoqun Dong, Zhuoyang Song, LiuYiBo LiuYiBo, Qianguoosun Qianguoosun, Yuxin Liang, Hao Wang, Enming Zhang, and Jiaxing Zhang. 2024. Never Lost in the Middle: Mastering Long-Context Question Answering with Position-Agnostic Decompositional Training. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13628–13642.
- [17] Cheng-Yu Hsieh, Yung-Sung Chuang, Chun-Liang Li, Zifeng Wang, Long Le, Abhishek Kumar, James Glass, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, et al. 2024. Found in the middle: Calibrating Positional Attention Bias Improves Long Context Utilization. In *Findings of the Association for Computational Linguistics ACL 2024*. 14982–14995.
- [18] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
- [19] Jakub Kuzilek, Martin Hlosta, and Zdenek Zdrahal. 2017. Open university learning analytics dataset. *Scientific data* 4, 1 (2017), 1–8.
- [20] Rui Li, Jianlin Su, Chenxi Duan, and Shunyi Zheng. 2020. Linear attention mechanism: An efficient attention for semantic segmentation. *arXiv preprint arXiv:2007.14902* (2020).
- [21] Yiming Lin, Madelon Hulsebos, Ruiying Ma, Shreya Shankar, Sepanta Zeigham, Aditya G Parameswaran, and Eugene Wu. 2024. Towards Accurate and Efficient Document Analytics with Large Language Models. *arXiv preprint arXiv:2405.04674* (2024).
- [22] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baille Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, and Gerardo Vitagliano. 2024. A Declarative System for Optimizing AI Workloads. *arXiv preprint arXiv:2405.14696* (2024).
- [23] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.
- [24] Shicheng Liu, Jialiang Xu, Wesley Tjanganaka, Sina J Semnani, Chen Jie Yu, and Monica S Lam. 2023. SUQL: Conversational Search over Structured and Unstructured Data with Large Language Models. *arXiv preprint arXiv:2311.09818* (2023).
- [25] Yusha Liu, Yichong Xu, Nihar B Shah, and Aarti Singh. 2022. Integrating rankings into quantized scores in peer review. *arXiv preprint arXiv:2204.03505* (2022).
- [26] Andrew Maas, Raymond E Daly, Peter T Pham, Dan Huang, Andrew Y Ng, and Christopher Potts. 2011. Learning word vectors for sentiment analysis. In *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies*. 142–150.
- [27] Arjun Panickssery, Samuel R Bowman, and Shi Feng. 2024. Llm evaluators recognize and favor their own generations. *arXiv preprint arXiv:2404.13076* (2024).
- [28] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. 2024. Lotus: Enabling semantic queries with llms over tables of unstructured and structured data. *arXiv preprint arXiv:2407.11418* (2024).
- [29] Erhard Rahm, Hanna Köpcke, and Andreas Thor. 2017. DBLP ACM Dataset. doi:10.3886/E100843V2
- [30] Keita Saito, Akifumi Wachi, Koki Wataoka, and Youhei Akimoto. 2023. Verbosity bias in preference labeling by large language models. *arXiv preprint arXiv:2310.10076* (2023).
- [31] Ashudeep Singh and Thorsten Joachims. 2018. Fairness of exposure in rankings. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 2219–2228.
- [32] Richard Sinkhorn. 1967. Diagonal equivalence to matrices with prescribed row and column sums. *The American Mathematical Monthly* 74, 4 (1967), 402–405.
- [33] Richard Sinkhorn and Paul Knopp. 1967. Concerning nonnegative matrices and doubly stochastic matrices. *Pacific J. Math.* 21, 2 (1967), 343–348.
- [34] Rickard Stureborg, Dimitris Alikanotis, and Yoshi Suhara. 2024. Large language models are inconsistent and biased evaluators. *arXiv preprint arXiv:2405.01724* (2024).
- [35] Zhiqing Sun, Yikang Shen, Qinhong Zhou, Hongxin Zhang, Zhenfang Chen, David Cox, Yiming Yang, and Chuang Gan. 2024. Principle-driven self-alignment of language models from scratch with minimal human supervision. *Advances in Neural Information Processing Systems* 36 (2024).
- [36] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. 2024. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118* (2024).
- [37] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).
- [38] Koki Wataoka, Tsubasa Takahashi, and Ryokan Ri. 2024. Self-preference bias in llm-as-a-judge. *arXiv preprint arXiv:2410.21819* (2024).
- [39] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2023. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864* (2023).
- [40] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. 2021. An explanation of in-context learning as implicit bayesian inference. *arXiv preprint arXiv:2111.02080* (2021).
- [41] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671* (2024).
- [42] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. 2020. Big bird: Transformers for longer sequences. *Advances in neural information processing systems* 33 (2020), 17283–17297.
- [43] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. 2024. DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence. *arXiv preprint arXiv:2406.11931* (2024).

Appendix

A Related Work

In this section, we give a brief literature review for this work. A more detailed literature review is provided in the Technical Report [9].

Handling Long Inputs: The challenge of *handling long input prompts* is a widely recognized issue in large language models. Various studies have approached this problem from multiple perspectives. One notable scenario occurs when prompts involve in-context learning [4, 5, 40], which often results in extended inputs that LLMs struggle to fully process. The “Lost in the Middle” phenomenon⁵, as identified by Liu et al. [23], highlights this issue, where LLMs fail to retain or utilize certain portions of lengthy inputs. Other studies have approached this issue by modifying the training datasets used for LLMs [16]. Several studies have explored modifying the architecture of LLMs, particularly the attention mechanisms, to improve their ability to process extensive context and handle larger input sizes [3, 7, 8, 17, 20, 42].

Debiasing LLM outputs: Part of our technical solution in Section 3 involves estimating input elements’ relevance to the query, where we learn a so-called bias coefficient for various LLM evaluations. Debiasing LLM outputs is extensively studied across various domains such as recommender systems rankings[6], using a variety of solutions such as output aggregation [13], prompt-writing [12], and many more [10, 27, 30, 34, 35, 38]. Note that despite their name similarity, the objective of these papers is different as they aim to mitigate the societal bias and unfairnesses introduced during various stages such as data-driven systems. Related work also includes reducing the output variance of LLMs using self-consistency methods [37], where the diversity of the model’s responses is leveraged to enhance reliability and accuracy. This approach is fundamentally different from our relevance estimation subproblem, where we aim to estimate the relevance of input items to the query.

B Proofs

THEOREM 1. *The process described in Equation 4 would eventually converge.*

PROOF. Define the matrix $W = (w_{i,j})$. Let $\tilde{S} = (\tilde{S}_i)$ be a vector of size n and $\beta = (\beta_j)$ be a vector of size $\sigma \cdot m$. For a vector $v = (v_i)$ of size n , let $\frac{1}{v}$ denote the new vector $(\frac{1}{v_1}, \frac{1}{v_2}, \dots, \frac{1}{v_n})$ and $v[k]$ denote the k^{th} element of a vector v .

Let $A \circ v$ denote the Hadamard product of an $n \times m$ matrix A to vector v of size m . The result of this production is a matrix B of size $n \times m$ where $B(i, j) = A(i, j) \cdot v[j]$. We also use $\mathbf{1}_n$ to denote a vector of size n with all elements equal to 1.

We can rewrite Equation 4 using the new notations:

$$\begin{aligned}\tilde{S}^{(T)} &= \frac{1}{\sigma} \cdot W \cdot \frac{1}{\beta^{(T-1)}} \\ \beta^{(T+1)} &= \frac{1}{\lceil \frac{n}{m} \rceil} \cdot W^\top \cdot \frac{1}{\tilde{S}^{(T)}}\end{aligned}$$

⁵While there have been attempts to resolve the lost in the middle issue in recent LLMs, as we shall experiment in Section 5.1, the attention has shifted to various sections of the input, and is not uniform across various input positions (see Figure 3).

Let us change the vector products to Hadamard products:

$$\tilde{S}^{(T)} = \frac{1}{\sigma} \cdot \left(W \circ \frac{1}{\beta^{(T-1)}} \right) \cdot \mathbf{1}_{\sigma \cdot m} \quad (6)$$

$$\beta^{(T+1)} = \frac{1}{\lceil \frac{n}{m} \rceil} \cdot \left(W^\top \circ \frac{1}{\tilde{S}^{(T)}} \right) \cdot \mathbf{1}_n \quad (7)$$

We can now define a sequence of matrices W_t according to the above update process:

$$W_t = \begin{cases} W \circ \frac{1}{\beta^{(t/2)}} & \text{if } t \text{ is even,} \\ \left(W^\top \circ \frac{1}{\tilde{S}^{((t+1)/2)}} \right)^\top & \text{if } t \text{ is odd.} \end{cases}$$

For example, $W_0 = W \cdot \frac{1}{\beta^{(0)}} = W$ because $\beta^{(0)} = \mathbf{1}_{\sigma \cdot m}$. However, W_1 is a column-scaled of W , so the sum of each column is now equal to σ . Subsequently, W_2 is a row-scaled of W_1 , so that the sum of each row is equal to $\lceil n/m \rceil$. This process continues similarly. At each iteration, we either rescale the rows or columns of W_t to get the matrix W_{t+1} . As a result, we can reduce this problem to the following:

Given a matrix W of size $n \times (\sigma m)$, at each iteration, we are alternatively scaling the rows to have a sum of σ or scaling the columns to have a sum of $\lceil n/m \rceil$. We would like to know the convergence condition for this process.

This is the same process as Sinkhorn’s algorithm [32, 33] to find a doubly stochastic matrix starting from a positive matrix W . If W is a non-negative matrix with at least one positive diagonal (refer to [33]), then this process would converge to a doubly stochastic matrix of W such that in this matrix (W_∞), the sum of each row is equal to σ and the sum of each column is equal to $\lceil n/m \rceil$. We skip most of the details to the references.

Now, we can rewrite the Equations 6 and 7 as:

$$\tilde{S}^{(\infty)} = \frac{1}{\sigma} \cdot W_\infty \cdot \mathbf{1}_{\sigma \cdot m} \quad (8)$$

$$\beta^{(\infty)} = \frac{1}{\lceil n/m \rceil} \cdot W_\infty \cdot \mathbf{1}_n \quad (9)$$

□

C Exposure Discovery Details

C.1 Estimation

Let $\pi_1, \pi_2, \dots, \pi_p$ be a set of p random permutations on $\mathcal{I}$. We apply each permutation π_j on $\mathcal{I}$ to obtain the permuted lists $\mathcal{I}_{\pi_j}$. This would change the position of token $t_{\pi_j(i)}$ to i . Within each permuted list, the relevant tokens are positioned at random locations. Subsequently, we generate the output of the large language model (LLM) for each $\mathcal{I}_{\pi_j}$ and compute the error $\epsilon_{\mathcal{L}}(\mathcal{I}_{\pi_j}, q)$. This process allows us to sample exposures according to the relationship defined in equation (5).

We then create an $n \times p$ matrix $\mathcal{R}$, such that $\mathcal{R}_{i,j} = \text{Rel}_q(t_{\pi_j(i)})$. Let $\vec{\epsilon}$ be a vector of size p such that $\vec{\epsilon}_j = \frac{1}{\epsilon_{\mathcal{L}}(\mathcal{I}_{\pi_j}, q)}$. Now, we should solve the following equation to find the unknown exposure vector $\mathcal{X}$:

$$\mathcal{R}^\top \cdot \mathcal{X} = \vec{\epsilon} \quad (10)$$

We can choose a value p larger than n to obtain a sufficient number of samples, thereby ensuring that the system of equations is overdetermined and can be solved effectively.

In order to solve equation (10), we find an estimation $\bar{\mathcal{X}}$ to minimize the Mean Squared Errors.

$$\begin{aligned} MSE(\bar{\mathcal{X}}) &= \|\mathcal{R}^\top \cdot \bar{\mathcal{X}} - \tilde{\epsilon}\|_2, \\ \nabla MSE(\bar{\mathcal{X}}) &= 0 \Leftrightarrow \bar{\mathcal{X}} = (\mathcal{R} \cdot \mathcal{R}^\top)^{-1} \cdot \mathcal{R} \cdot \tilde{\epsilon} \end{aligned}$$

As a result, $\bar{\mathcal{X}}_i$ would be an estimation for $\mathcal{X}_{\mathcal{L}}(i)$.

C.2 Confidence

While one can use a fixed budget on the number of permutations to use for estimating the exposure values, the user can alternatively specify a target variance in the estimation. In such cases, considering the exponential search strategy, we start from a base number of permutations and compute the estimation variance, as explained in the following. If the estimation variance is larger than the target variance, we double the number of permutations (i.e., double the value of p) and repeat the process.

We follow the standard confidence interval analysis for a Mean Squared Error estimation. Each $\bar{\mathcal{X}}_i$ is an unbiased estimator of $\mathcal{X}_{\mathcal{L}}(i)$. This estimator follows a t-Distribution around the real value:

$$\frac{\bar{\mathcal{X}}_i - \mathcal{X}_{\mathcal{L}}(i)}{\sqrt{Var(\bar{\mathcal{X}}_i)}} \sim t_{p-n} \quad (11)$$

Where t_{p-n} is the t-Distribution with $p - n$ degree of freedom and,

$$Var(\bar{\mathcal{X}}_i) = \hat{\sigma}^2 \cdot [(\mathcal{R} \cdot \mathcal{R}^\top)^{-1}]_{i,i}, \quad (12)$$

$$\hat{\sigma} = \frac{\|\mathcal{R}^\top \cdot \bar{\mathcal{X}} - \tilde{\epsilon}\|_2}{p - n} \quad (13)$$

By increasing the number of random permutations p , we increase the degree of freedom and as a result we would have less variance and a narrower distribution for the estimation.

C.3 Practical Concerns

The process for exposure value estimation requires p separate API calls to the large language model, $\mathcal{L}$. In the following, we explore heuristic methods aimed at reducing the number of required API calls, thereby optimizing the associated costs.

At first, we can choose a task (I, q) such that there is only one token t_i related to the query q . As a result, one can just place this relevant element into different positions and sample from each $\mathcal{X}_{\mathcal{L}}(i)$ individually. Secondly, one can carefully create a task such that the result identifies how much information from all the positions is retained by the LLM.

Specifically, consider the *token-counting task* that counts the number of occurrences of each token in the input I . For example, given the input $I = \{a, b, b, b, a, a, a, c, c\}$, the output is “a: 4, b: 3, c: 2”. Note that in this example, all elements are relevant to the query, but each of them is relevant to one of the sub-problems (contributing only to one of the token counts). Using this technique, each query to the LLM provides relevant information about *all* rank positions (not only a subset of positions that are relevant to the query). We shall discuss this technique further with more examples in our experiments (Section 5).

So far in this section, we examined the exposure at each token position within the input prompt. However, for specific tasks each element $e \in \mathcal{I}$ can correspond to multiple tokens. Let $\ell(e)$ denote the number of tokens associated with element e . As outlined in Section 2.4, the objective is to compute the utility of a particular ranking, which depends on the exposure of entire elements, rather than individual tokens, within the reranked input. To generalize the definition of utility, we define the exposure of each element e as the average exposure of the tokens that compose it. In other words, we rewrite equation 2 as,

$$\mathbb{E}[\text{utility}(\pi \mid q)] = \sum_{i=1}^{|I|} \left(\mathbb{E} \left[\frac{\sum_{j=i}^{i+\ell(e_i)} \mathcal{X}_{\mathcal{L}}(j)}{\ell(e_i)} \right] \cdot \mathbb{E}[\text{Rel}_q(e_{\pi_i})] \right)$$